

Boletín de ejercicios de Dockerfile (con solucións)

1. Comando básico de bash dentro dun contedor

Obxectivo: crear unha imaxe mínima que execute un comando de shell ao iniciar. Instrucións:

1. Emprega a imaxe base `alpine:3.20`.
2. Fai que o contedor execute un comando que liste o contido do directorio raíz e, a continuación, amose os procesos en execución.
3. Constrúe a imaxe co nome `bash-mini:ls-ps` e execútaa.

Solución (Dockerfile):

```
FROM alpine:3.20
CMD ["sh", "-c", "ls -la / && echo '---' && ps aux"]
```

Construír e executar:

```
docker build -t bash-mini:ls-ps .
docker run --rm bash-mini:ls-ps
```

2. Servidor web con Nginx

Obxectivo: servir un HTML estático. Instrucións:

1. Usa a imaxe base `nginx:alpine`.
2. Crea un `index.html` con o texto "Benvido a Nginx en Docker" e cópiao a `/usr/share/nginx/html/` no contedor.
3. Expón o porto `80` no Dockerfile.
4. Constrúe a imaxe `web:nginx` e lánzaa mapeando o porto `8080:80` do host.

Solución (estrutura mínima):

```
/proyecto
├── Dockerfile
└── index.html
```

`index.html` (exemplo):

```
<!doctype html>
<html><body><h1>Benvido a Nginx en Docker</h1></body></html>
```

Dockerfile:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Construír e executar:

```
docker build -t web:nginx .
docker run --rm -p 8080:80 --name web web:nginx
# Navega a http://localhost:8080
```

3. Aplicación Flask con requirements.txt

Obxectivo: levantar un pequeno servidor Flask. Instrucións:

1. Crea `requirements.txt` cunha dependencia: `Flask==3.0.0`.
2. Crea `app.py` cun endpoint raíz que devolva "Ola desde Flask e Docker!".
3. No Dockerfile: usa `python:3.12-slim`, copia e instala `requirements.txt`, copia o código, expón `5000` e define `CMD` para executar a app.
4. Constrúe a imaxe `flask:web` e execútaa mapeando `5000:5000`.

Solución (estrutura mínima):

```
/proyecto
├── Dockerfile
├── requirements.txt
└── app.py
```

requirements.txt:

```
Flask==3.0.0
```

app.py:

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def home():
    return "Ola desde Flask e Docker!"
```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile:

```
FROM python:3.12-slim  
  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY . .  
  
EXPOSE 5000  
CMD ["python", "app.py"]
```

Construír e executar:

```
docker build -t flask:web .  
docker run -d --rm -p 5000:5000 --name flaskweb flask:web  
# Probar: curl http://localhost:5000/
```

4. Versión avanzada con script de inicio, usuario non root e HEALTHCHECK

Obxectivo: integrar boas prácticas. Instrucións:

1. Base: `python:3.12-slim`.
2. Crea `start.sh` que mostre unha mensaxe e execute o comando recibido.
3. Engade un usuario `appuser` e executa a app con el.
4. Instala dependencias desde `requirements.txt` (podes reutilizar o do exercicio 3).
5. Engade un `HEALTHCHECK` que faga unha petición HTTP ao porto da app cada 30s.
6. Constrúe a imaxe `flask:secure` e execútaa en segundo plano mapeando `8000:8000`.

Solución (estrutura mínima):

```
/proyecto  
├── Dockerfile  
├── requirements.txt  
├── app.py  
└── start.sh
```

requirements.txt (exemplo):

```
Flask==3.0.0
unicorn==21.2.0
```

app.py:

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def home():
    return "App segura con usuario non root!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

start.sh:

```
#!/usr/bin/env bash
set -euo pipefail
echo "[start] Iniciando app..."
exec "$@"
```

Dockerfile:

```
FROM python:3.12-slim

# Crear usuario non root
RUN addgroup --system app && adduser --system --ingroup app app

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py start.sh ./
RUN chmod +x start.sh && chown -R app:app /app

USER app

EXPOSE 8000

# Healthcheck (precisa curl; en python slim non está por defecto, úsase wget)
HEALTHCHECK --interval=30s --timeout=3s --retries=3 CMD wget -qO-
http://127.0.0.1:8000/ >/dev/null 2>&1 || exit 1

ENTRYPOINT ["/start.sh"]
```

```
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app", "--workers", "2", "--threads",  
"2"]
```

Construir e executar:

```
docker build -t flask:secure .  
docker run -d --rm -p 8000:8000 --name flasksecure flask:secure  
# Probar: curl http://localhost:8000/  
# Ver estado health: docker inspect --format='{{json .State.Health}}' flasksecure
```